

Quotation Comparison Logic Handoff

Document Purpose

This document explains the current Procurement Advisory implementation in the Procurement Assistant prototype. It is intended for the next developer or Codex session to understand how quotation upload, extraction, comparison, scoring, recommendation, and Decision Summary generation work today.

The implementation is demo-only and deterministic. It does not call OpenAI, any external AI API, any document-processing API, or any email service.

1. Overall Procurement Advisory Workflow

The Procurement Advisory flow starts from the landing page and routes the user to `/advisory`.

Text-based workflow:

```
Landing page (/)
-> User selects Procurement Advisory
-> Advisory page (/advisory)
-> User selects or drops quotation files
-> UI stores selected File objects in component state
-> UI requires at least two files before comparison
-> User clicks Compare
-> Loading banner: "Reviewing quotations..."
-> analyzeQuotationFiles(files)
-> extractQuotationSummary(input) for each file
-> compareQuotations(summaries)
-> Decision Summary
-> Vendor quotation cards sorted by recommendation score
```

The advisory page is implemented in `app/advisory/page.tsx`. The deterministic quotation logic is implemented in `lib/agent/quotationAgent.ts`.

2. Complete Quotation Upload Flow

The upload workflow is fully client-side and local to `app/advisory/page.tsx`.

Main state:

- `selectedFiles`: array of selected quotation files with local IDs.
- `comparison`: generated `QuotationComparison`, or `null` before results.
- `reviewing`: controls loading state.
- `dragActive`: controls drop-area visual state.
- `validationMessage`: stores compact validation feedback.

Upload behavior:

1. User selects files through a hidden `<input type="file" multiple />` or drops files onto the upload area.

2. `handleFiles(files)` converts the `FileList` into `SelectedQuotationFile[]`.
3. Each selected file receives a local UI ID from `createFileId(file, index)`.
4. Duplicate IDs are ignored.
5. The file input value is reset so the same file can be selected again after removal.
6. The current comparison result is cleared whenever files change.
7. Selected files are shown in a compact responsive list.
8. Each selected file can be removed with `removeFile(id)`.
9. The Compare button is disabled until at least two files are selected.
10. If fewer than two files are selected, the UI shows: Upload at least two quotations.

Comparison trigger:

1. `handleCompare()` checks that at least two files are selected.
2. If fewer than two files are selected, it keeps the validation message and exits.
3. If valid, it clears previous results, sets `reviewing` to `true`, and starts a short `setTimeout`.
4. After the delay, it calls `analyzeQuotationFiles(selectedFiles.map(item => item.file))`.
5. The returned comparison is stored in state and displayed.

The loading state is intentionally minimal:

```
Reviewing quotations...
```

3. Complete Quotation Extraction Logic

Quotation extraction is handled by `extractQuotationSummary(input)` in `lib/agent/quotationAgent.ts`.

The extraction pipeline:

```
File
-> file.text()
-> sanitizeReadableText()
-> QuotationInput
-> normalizeText()
-> extractVendorName()
-> extractProducts()
-> extractQuantity()
-> extractPriceCandidates()
-> selectTotalPrice()
-> detectCurrency()
-> extractDelivery()
-> extractWarranty()
-> extractSupport()
-> extractPaymentTerms()
-> extractValidity()
-> extractTaxes()
-> extractFreight()
-> extractInstallation()
-> extractInclusionsExclusions()
-> extractComplianceIndicators()
-> extractTechnicalFit()
-> getMissingCommercialInfo()
-> buildInitialRisks()
-> buildInitialPros()
-> buildInitialCons()
-> buildNegotiationLevers()
-> QuotationSummary
```

`analyzeQuotationFiles(files)` attempts to read every file with `file.text()`. If reading fails, the text is set to an empty string. This keeps unsupported or unreadable files from crashing the UI. The parser then falls back to filename and metadata where possible.

Readable text is sanitized by `sanitizeReadableText(rawText)`:

- Empty text stays empty.
- Files with a high proportion of control characters are treated as unreadable.
- Carriage returns are normalized to newlines.
- Repeated whitespace is normalized.
- Text is capped at `READABLE_TEXT_LIMIT`, currently 120000 characters.

All missing procurement-critical fields are represented as `Needs clarification` or added to missing/risk lists.

4. Vendor Extraction Logic

Vendor extraction is handled by `extractVendorName(text, fileName)`.

The function checks in this order:

1. Explicit vendor lines in the first 60 meaningful lines, using labels such as:
 - vendor
 - supplier
 - company
 - seller
 - bidder
 - reseller
 - partner
 - quoted by
 - quotation from
 - quote from
 - from
2. Submission/preparation labels such as:
 - prepared by
 - submitted by
 - issued by
 - commercial offer by
3. A phrase where a candidate appears before quotation, quote, proposal, or commercial offer.
4. Header fallback from the first 12 meaningful lines.
5. Filename fallback after removing common quotation words.
6. `Needs clarification` if no useful value exists.

Validation rules:

- Explicit vendor names must be 2 to 90 characters.
- Generic commercial words such as quotation, payment, delivery, warranty, total, amount, and price are rejected.
- Non-explicit vendor names must look like likely company names, using terms such as Pvt, Limited, Ltd, LLP, Inc, Technologies, Systems, Solutions, Enterprises, Computers, Infotech, Services, Distributors, or Resellers.

5. Product Extraction Logic

Product extraction is handled by `extractProducts(text, fileName)`.

It uses two main approaches:

1. Brand/model pattern matching.
2. Procurement line extraction.

Recognized product/model families include:

- Dell Latitude, Precision, Vostro, XPS, OptiPlex
- HP EliteBook, ProBook, ZBook, Pavilion, Dragonfly
- Lenovo ThinkPad, ThinkBook, Yoga, Legion, IdeaPad
- Apple MacBook
- Acer TravelMate, Swift, Aspire
- Asus ExpertBook, Zenbook, Vivobook

Line-based extraction looks for lines containing terms such as:

- product
- model
- item
- description
- make
- part no
- sku
- laptop
- notebook
- workstation
- configuration
- specification

It excludes lines that are clearly commercial rather than product-related, such as payment, delivery, warranty, tax, GST, freight, shipping, total, or subtotal.

If no product is found but the filename contains laptop, notebook, or workstation, the product fallback is:

Laptop quotation

The result is limited to four product entries.

6. Commercial Comparison Logic

Commercial comparison is spread across extraction, scoring, risk generation, and recommendation finalization.

Commercial fields extracted:

- Quoted total price
- Currency
- Quantity
- Unit price
- Taxes
- Freight
- Installation/configuration
- Payment terms
- Quote validity
- Inclusions/exclusions
- Missing commercial information

Important functions:

- `extractPriceCandidates()`
- `selectTotalPrice()`
- `extractTaxes()`
- `extractFreight()`
- `extractInstallation()`
- `extractPaymentTerms()`
- `extractValidity()`
- `getMissingCommercialInfo()`
- `buildInitialRisks()`
- `buildNegotiationLevers()`
- `scoreCommercialTerms()`
- `hasCommercialRisk()`

The comparison logic treats hidden or unclear costs as procurement risks. A cheaper quotation can be downgraded when taxes, freight, payment terms, or key fields are unclear.

7. Price Comparison Logic

Price extraction is handled by `extractPriceCandidates(text)`.

Recognized currencies:

- INR

- Rs
- Rs .
- rupee symbol
- USD
- dollar symbol
- EUR
- euro symbol

Price candidate extraction:

- Scans meaningful lines.
- Finds currency-prefixed or currency-suffixed amounts.
- Finds amount-only values when the line is in a price context.
- Normalizes amounts by removing commas and rounding.
- Classifies each price candidate as `total`, `unit`, or `other`.
- Assigns a priority based on commercial wording.

Candidate kind rules:

- `unit`: unit price, unit rate, rate per, per unit, unit cost, each.
- `total`: grand total, total quoted, total amount, total price, total cost, net amount, landed cost, quote value, final total, payable, amount payable, order value, commercial value, subtotal.
- `other`: price-like values without a stronger label.

Priority rules:

Priority	Context
5	grand total, final total, amount payable, landed cost, net payable
4	total quoted, total amount, total price, total cost, quote value, order value
3	general price context
2	subtotal, before tax, taxable value
1	unit price, unit rate, per unit, rate per, each

Total price selection:

1. Prefer candidates classified as `total`.
2. If no total exists, prefer non-unit candidates.
3. If quantity exists and multiple candidates exist, use the highest value as likely total.
4. If only unit price and quantity are known, total is calculated as `quantity * unitPrice`.

Price scoring:

`scorePrice(quotation, lowestPriceValue)` gives up to 30 points.

- Missing price: 2 points.

- Otherwise: `round(clamp(30 * (lowestPrice / quotePrice), 10, 30))`.
- If taxes or freight are extra, subtract 3 points, with a minimum of 8.

This means lowest price can score well, but unclear landed cost reduces the score.

8. Tax Comparison Logic

Tax extraction is handled by `extractTaxes(text)`.

Returns:

- Included
- Extra
- GST n%
- Needs clarification

Included indicators:

- inclusive of GST
- GST included
- inclusive of taxes
- taxes included
- including GST
- incl. GST
- tax inclusive

Extra indicators:

- exclusive of GST
- GST extra
- taxes extra
- tax extra
- exclusive of taxes
- plus GST
- GST additional
- taxes additional
- tax exclusive

If a GST or tax percentage is found, the result is stored as GST n%.

Commercial impact:

- Included taxes improve commercial score.
- Extra taxes create risk and cons.
- Unknown taxes add missing commercial information and negotiation levers.

9. Delivery Comparison Logic

Delivery extraction is handled by `extractDelivery(text)`.

Immediate stock handling:

- ex-stock
- ready stock
- in stock
- available now
- immediate delivery

These return:

```
label: Ready stock
days: 2
```

Pattern-based extraction recognizes:

- delivery in N days/weeks/months
- lead time N days/weeks/months
- N days/weeks/months delivery
- within/in N days/weeks/months from PO, purchase order, order, advance, or payment

`toDays(value, unit)` normalizes:

- weeks to value * 7
- months to value * 30
- days to value

Delivery scoring:

Delivery days	Score
Missing	3
<= 7	15
<= 14	12
<= 30	9
> 30	5

Fastest delivery is selected with `minBy()` over quotations with known delivery days.

10. Warranty Comparison Logic

Warranty extraction is handled by `extractWarranty(text)`.

Recognized patterns:

- warranty N years/months
- onsite warranty N years/months
- support warranty N years/months
- OEM warranty N years/months
- N years/months warranty
- N years/months onsite/support

Warranty months are normalized by `toMonths()`:

- years to value * 12
- months to value

Support/SLA extraction is handled by `extractSupport(text)`.

Support signals include:

- 24x7 support
- 24/7 support
- next business day
- NBD
- onsite support
- on-site support
- SLA
- RMA
- replacement support
- escalation
- service desk
- helpdesk
- back-to-back OEM support

Warranty/support scoring:

Condition	Score
Missing warranty, no support	3
Missing warranty, support present	6
Warranty >= 36 months and support present	15
Warranty >= 36 months	13
Warranty >= 12 months and support present	11
Warranty >= 12 months	8
Warranty below 12 months	5

Strongest warranty is selected with `maxBy()` over known warranty months.

11. Payment Terms Comparison

Payment extraction is handled by `extractPaymentTerms(text)`.

Recognized patterns:

- payment terms: ...
- payment: ...
- commercial terms: ...
- billing terms: ...
- terms of payment: ...
- net N days
- credit N days
- N% advance
- N% upfront
- against delivery
- on delivery
- after delivery
- against PO
- on purchase order

Payment impact:

- Stated payment terms improve commercial score.
- Advance or upfront payment terms score lower.
- Credit, net, after delivery, on delivery, or against delivery terms improve score.
- Missing payment terms add missing commercial information, cons, risks, and negotiation levers.

12. Compliance Comparison

Compliance extraction is handled by `extractComplianceIndicators(text)`.

Recognized indicators:

- OEM Authorized Partner
- OEM Authorised Partner
- Authorized reseller
- Authorised reseller
- Authorized partner
- Authorised partner
- OEM partner
- Certified partner
- GST Registered

- GSTIN
- ISO 9001
- ISO 27001
- MSME
- GeM seller
- RoHS
- EPEAT
- Energy Star

If OEM Authorized Partner is found, the generic Authorized partner duplicate is removed. The same applies to the British spelling variant.

Compliance scoring:

Condition	Score
3 or more indicators and authorization/certification present	10
2 or more indicators	8
1 indicator with authorization/certification	6
1 other indicator	5
No indicators	2

13. Risk Identification Logic

Risk generation starts in `buildInitialRisks(fields)` and is later refined in `finalizeRecommendation()`.

Initial risks:

- Every missing commercial field becomes `<field> missing`.
- Taxes extra becomes `Taxes excluded`.
- Freight extra becomes `Freight excluded`.
- Advance/upfront payment becomes `Advance payment requirement`.
- Missing delivery becomes `Delivery commitment unclear`.
- Missing warranty becomes `Warranty commitment unclear`.
- If no major gaps exist, fallback is `No major gaps detected`.

Additional finalization:

- If any missing commercial information exists, the first missing item is added as `<field> is missing`.
- High missing information can force the recommendation tag to `Needs clarification`.
- Taxes extra, freight extra, advance/upfront payment, or many missing fields can force `Commercial risk`.

14. Pros Generation

Pros are generated by `buildInitialPros(fields)` and refined by `finalizeRecommendation()`.

Initial pros:

- Price provided
- Delivery stated
- Warranty stated
- Support details present
- Commercial terms stated
- Taxes included
- Freight included
- Configuration included
- Compliance indicators present

Fallback:

Quotation received

Final pros may also include:

- Lowest quoted total
- Fastest delivery
- Strong warranty position
- Strongest value balance

The final list is capped at five items.

15. Cons Generation

Cons are generated by `buildInitialCons(fields)` and refined by `finalizeRecommendation()`.

Initial cons:

- First three missing commercial fields become `<field> not clear`.
- Taxes extra becomes `Taxes extra`.
- Freight extra becomes `Freight extra`.
- Advance/upfront payment becomes `Advance payment`.

Fallback:

No major concern detected

Finalization adds:

`<first missing commercial field> needs clarification`

The final list is capped at five items.

16. Vendor Scoring Logic

Scoring is handled by `scoreQuotation(quotation, lowestPriceValue)`.

Total score is 100 points.

Scoring table:

Dimension	Max points	Function
Price competitiveness	30	<code>scorePrice()</code>
Requirement / product fit	15	<code>scoreProductFit()</code>
Delivery and fulfillment	15	<code>scoreDelivery()</code>
Warranty and support	15	<code>scoreWarranty()</code>
Commercial terms	10	<code>scoreCommercialTerms()</code>
Compliance / reliability	10	<code>scoreCompliance()</code>
Completeness / low ambiguity	5	inline in <code>scoreQuotation()</code>
Total	100	<code>scoreQuotation()</code>

Product fit scoring:

Condition	Score
Aligned, product present, quantity present	15
Aligned	12
Equivalent offered with product present	10
Equivalent offered without product present	8
Product present, fit unclear	7
Product missing, fit unclear	4

Commercial terms scoring:

- Payment terms stated: up to 3 points.
- Advance/upfront payment terms receive only 1 point for the payment component.
- Quote validity stated: 2 points.
- Taxes included: 3 points.
- Taxes known but not included: 1 point.
- Freight included: 1 point.
- Credit/net/after-delivery/on-delivery terms: 2 extra points.
- Final commercial score is clamped to 10.

Completeness scoring:

```
max(0, 5 - min(5, missingCommercialInfo.length))
```

This penalizes ambiguity directly.

17. Recommendation Logic

Recommendation logic is handled by `compareQuotations()` and `finalizeRecommendation()`.

Comparison markers:

- `lowestPrice`: quotation with lowest known total price.
- `fastestDelivery`: quotation with lowest known delivery days.
- `strongestWarranty`: quotation with highest known warranty months.
- `bestValue`: highest-scoring decision-ready quotation.

Decision-ready criteria:

```
quotedTotalPrice is known
recommendationScore >= 55
missingCommercialInfo.length <= 3
productsOffered.length > 0
```

Recommendation tag priority:

1. If the quotation is best value: `Best value`.
2. Else if it has four or more missing commercial fields or no price: `Needs clarification`.
3. Else if it has commercial risk: `Commercial risk`.
4. Else if it is lowest price: `Lowest price`.
5. Else if it is fastest delivery: `Fastest delivery`.
6. Else if it has strongest warranty: `Strong warranty`.
7. Else: `Comparable option`.

Commercial risk criteria:

- Taxes extra.
- Freight extra.
- Advance/upfront payment.
- Three or more missing commercial fields.

The logic explicitly avoids choosing the cheapest vendor automatically. Best value requires acceptable completeness, known price, product presence, and a stronger total score.

18. Decision Summary Generation

Decision Summary is displayed by `DecisionSummary()` in `app/advisory/page.tsx` using the `QuotationComparison` object.

Displayed summary metrics:

- Best value
- Lowest price

- Fastest delivery
- Clarification
- Negotiate
- Key decision note

Source logic:

- bestValue: selected in compareQuotations().
- lowestPrice: selected in compareQuotations().
- fastestDelivery: selected in compareQuotations().
- topClarificationNeeded: generated by findTopClarification().
- topNegotiationLever: generated by findTopNegotiationLever().
- keyDecisionNote: generated by buildDecisionNote().

Decision note rules:

- If no quotation is decision-ready:
- No quotation is decision-ready. Clarify commercial gaps before award.
- If best value exists but no lowest-price marker exists:
- <vendor> has the strongest available score, but quoted price needs clarification.
- If best value is also lowest price:
- <vendor> leads on value and price. Validate final landed cost.
- If best value differs from lowest price:
- <vendor> offers stronger value than the lowest-price quote after delivery, warranty, terms, and risk.

Top clarification:

1. First missing commercial field from the sorted quotation list.
2. If none, first non-trivial risk.
3. Else None.

Top negotiation lever:

1. First negotiation lever from the top-scored quotation.
2. Else None.

19. Deterministic Rules Used

The advisory logic is deterministic because:

- It reads files locally in the browser.
- It uses regex and stable string rules.
- It uses fixed scoring weights.
- It uses stable fallback text.

- It uses local sorting by numeric score.
- It does not call external services.
- It does not use randomness.
- It does not use model-generated text.

Stable identifiers:

- UI file IDs use filename, size, lastModified, and selected index.
- Quotation IDs use a simple deterministic hash of filename, size, lastModified, and index.

Fallback behavior:

- Unreadable files produce empty text.
- Empty text still creates a `QuotationSummary`.
- Missing values appear as `Needs clarification`.
- Filename can provide vendor or product fallback.
- Missing commercial fields are surfaced as risks, cons, and clarification items.

20. Important Assumptions

Implementation assumptions:

- `BrowserFile.text()` is sufficient for demo-readable files.
- Some PDFs, Office files, email files, and binary files may not produce meaningful text in the browser.
- The prototype should accept those files without crashing.
- The UI should not claim production-grade extraction.
- Procurement users need a decision aid, not a raw parser report.
- Lowest price is only one factor.
- Clarified landed cost is more important than nominal quoted price.
- Warranty, support, delivery, compliance, and commercial clarity materially affect vendor decision quality.

Procurement assumptions:

- Laptop procurement is the primary use case.
- Product fit can be inferred from laptop model families and common technical terms.
- Missing delivery, warranty, taxes, payment, validity, or product detail should block confident award.
- Extra taxes/freight and advance payment are commercial risks.
- OEM authorization, GST clarity, and certifications improve confidence.

21. Current Limitations

Current limitations:

- No OCR.

- No server-side document parsing.
- No PDF table extraction.
- No Office document parser.
- No email attachment parser.
- No production-grade vendor normalization.
- No currency conversion.
- No multi-currency comparison adjustment.
- No tax-inclusive landed-cost calculation when taxes are only partially stated.
- No requirement document cross-check beyond simple product-fit signals.
- No persisted advisory session.
- No export from the advisory page.
- No manual correction workflow for extracted fields.
- No weighting configuration in the UI.
- Regex extraction may miss unusual quotation formats.
- Amount selection can be wrong when documents contain many unrelated financial numbers.

These limitations are acceptable for the current demo-only prototype.

22. Future Enhancement Opportunities

Useful future enhancements that preserve the current procurement-head workflow:

- Add manual correction of extracted quotation fields before comparison.
- Add configurable scoring weights for price, delivery, support, compliance, and completeness.
- Add export to PDF or CSV for the final comparison summary.
- Add a demo requirement baseline so quotation product fit can be compared against a selected requirement.
- Add deterministic table parsing for CSV and pasted text.
- Add optional local-only parsers for common document formats if dependency size remains acceptable.
- Add side-by-side comparison table for selected vendors.
- Add negotiation checklist generation from extracted risks.
- Add landed-cost calculation when tax/freight percentages are available.
- Add currency normalization only if exchange rates are provided locally or manually.
- Add saved advisory sessions in localStorage.

Avoid adding:

- OpenAI API.
- External AI APIs.
- External document-processing services.
- Real email sending.
- API keys.
- Real authentication.

23. All Important Functions Involved

Primary route/UI functions in `app/advisory/page.tsx`:

Function/component	Purpose
<code>AdvisoryPage()</code>	Main route component for upload, validation, loading, and results.
<code>handleFiles()</code>	Adds selected/dropped files, avoids duplicates, updates validation, clears old comparison.
<code>removeFile()</code>	Removes a selected file and updates validation.
<code>handleCompare()</code>	Validates file count, shows loading state, calls quotation analysis, stores comparison.
<code>DecisionSummary()</code>	Displays best value, lowest price, fastest delivery, clarification, negotiation, and decision note.
<code>QuotationCard()</code>	Displays each vendor quotation result.
<code>DecisionMetric()</code>	Renders one summary metric.
<code>InfoBlock()</code>	Renders a field/value pair inside a quotation card.
<code>ScoreBreakdown()</code>	Displays score dimensions and progress bars.
<code>ListBlock()</code>	Displays pros, cons, risks, and negotiation levers.
<code>formatPrice()</code>	Formats total price or clarification text.
<code>formatUnitPrice()</code>	Formats unit price or clarification text.
<code>formatAmount()</code>	Uses <code>Intl.NumberFormat("en-IN")</code> for amounts.
<code>formatOptionalNumber()</code>	Displays number or Needs clarification.
<code>formatList()</code>	Joins list values or displays Needs clarification.
<code>formatFileSize()</code>	Displays file size in B, KB, or MB.
<code>createFileId()</code>	Creates local UI file ID.

Primary exported logic in `lib/agent/quotationAgent.ts`:

Function/type	Purpose
<code>QuotationInput</code>	Normalized input object for one uploaded quotation.
<code>QuotationSummary</code>	Extracted, scored, and display-ready vendor quotation.
<code>QuotationComparison</code>	Overall comparison result for all quotations.
<code>QuotationScoreBreakdown</code>	Scoring dimensions.
<code>RecommendationTag</code>	Allowed recommendation labels.
<code>analyzeQuotationFiles()</code>	Reads browser files and runs extraction/comparison.
<code>extractQuotationSummary()</code>	Extracts procurement fields and initial pros/cons/risks.
<code>compareQuotations()</code>	Selects markers, scores vendors, finalizes recommendations, sorts results.

Internal extraction functions:

Function	Purpose
<code>extractVendorName()</code>	Finds vendor from explicit labels, header text, or filename.

Function	Purpose
extractProducts()	Finds product/model lines and known laptop model families.
extractQuantity()	Finds quantity/unit counts.
extractPriceCandidates()	Finds currency/amount candidates.
selectTotalPrice()	Chooses the most likely total price.
extractDelivery()	Extracts lead time and normalized days.
extractWarranty()	Extracts warranty label and normalized months.
extractSupport()	Finds support/SLA signals.
extractPaymentTerms()	Extracts payment terms.
extractValidity()	Extracts quote validity.
extractTaxes()	Classifies tax treatment.
extractFreight()	Classifies freight/shipping treatment.
extractInstallation()	Classifies installation/configuration treatment.
extractInclusionsExclusions()	Flags accessories, deployment services, and exclusions.
extractComplianceIndicators()	Extracts certification, authorization, and compliance signals.
extractTechnicalFit()	Classifies product fit as aligned, equivalent, or unclear.

Internal scoring/recommendation functions:

Function	Purpose
scoreQuotation()	Builds total score and scoring breakdown.
scorePrice()	Scores price competitiveness against lowest known price.
scoreProductFit()	Scores product/requirement fit.
scoreDelivery()	Scores delivery speed.
scoreWarranty()	Scores warranty and support.
scoreCommercialTerms()	Scores payment, validity, taxes, freight, and credit terms.
scoreCompliance()	Scores compliance/reliability indicators.
finalizeRecommendation()	Adds comparison-aware pros/risks/levers and assigns recommendation tag.
isDecisionReadyQuotation()	Filters vendors eligible for best value.
hasCommercialRisk()	Detects commercial risk conditions.
buildDecisionNote()	Generates concise decision note.
findTopClarification()	Finds the top clarification item.
findTopNegotiationLever()	Finds the top negotiation lever.

Internal utility functions:

Function	Purpose
getMissingCommercialInfo()	Lists missing procurement-critical fields.
buildInitialRisks()	Creates initial risk list.

Function	Purpose
buildInitialPros()	Creates initial pro list.
buildInitialCons()	Creates initial con list.
buildNegotiationLevers()	Creates negotiation actions.
isLikelyVendorName()	Validates inferred vendor candidates.
isValidExplicitVendorName()	Validates explicit vendor candidates.
getMeaningfulLines()	Splits and filters usable text lines.
sanitizeReadableText()	Cleans browser file text and rejects likely binary content.
normalizeText()	Normalizes whitespace.
cleanCandidate()	Cleans extracted text candidates.
toTitleCase()	Formats filename fallback vendor names.
parseAmount()	Parses numeric amount strings.
getPriceKind()	Classifies price candidate type.
getPricePriority()	Assigns price candidate priority.
isPriceContext()	Detects lines likely to contain commercial amounts.
detectCurrency()	Detects INR, USD, or EUR.
normalizeCurrency()	Normalizes currency symbols/codes.
toDays()	Converts weeks/months to approximate days.
toMonths()	Converts years to months.
normalizeDurationUnit()	Formats duration labels.
createStableId()	Creates deterministic quotation ID.
minBy()	Selects minimum by numeric selector.
maxBy()	Selects maximum by numeric selector.
addUnique()	Appends unique list entries.
clamp()	Restricts scores to allowed range.

24. Important Files Involved

File dependency diagram:

```

app/page.tsx
  -> Link to /advisory

app/advisory/page.tsx
  -> components/app-shell.tsx
  -> lib/agent/quotationAgent.ts
  -> lucide-react icons
  -> app/globals.css component classes

components/app-shell.tsx
  -> components/branded-background.tsx
  -> Next.js Link
  -> lucide-react Cpu icon

app/layout.tsx

```

```

-> app/globals.css
-> metadata: Procurement Assistant

lib/agent/quotationAgent.ts
-> browser File API through analyzeQuotationFiles(files)
-> no app data files
-> no backend APIs
-> no external services

```

Important file list:

File	Role
app/advisory/page.tsx	Main Procurement Advisory route, upload UI, validation, loading, Decision Summary, cards.
lib/agent/quotationAgent.ts	Deterministic extraction, comparison, scoring, recommendation, risks, pros, cons.
components/app-shell.tsx	Shared page shell and navigation used by advisory page.
components/branded-background.tsx	Shared visual background wrapper.
app/globals.css	Shared Tailwind component classes such as premium-card, primary-button, secondary-button, field-label.
app/page.tsx	Landing page that routes users to Procurement Advisory.
app/layout.tsx	Root layout and metadata.
package.json	Confirms framework and dependencies: Next.js, React, TypeScript, Tailwind, lucide-react.

Developer Explanation

The Procurement Advisory implementation is intentionally compact. The page component owns the interaction state and display. The quotation agent owns all business logic. This separation keeps UI behavior and deterministic procurement evaluation loosely coupled.

The page does not parse files directly beyond passing browser File objects to `analyzeQuotationFiles()`. The agent reads file text, normalizes it, extracts procurement fields with regex and fixed rules, produces `QuotationSummary` objects, scores them, identifies comparison markers, and returns a sorted `QuotationComparison`.

The most important developer constraint is to keep extraction and scoring deterministic. Any improvement should be expressed as explicit parsing rules, fixed scoring rules, or transparent fallback behavior. Avoid adding remote services, hidden model calls, or behavior that cannot be explained from the source code.

When extending the feature, preserve these boundaries:

- UI: file selection, validation, loading, display, formatting.
- Agent: text reading, extraction, comparison, scoring, recommendation.
- Styling: existing Tailwind classes and restrained enterprise presentation.

Procurement Manager Explanation

The Procurement Advisory page helps compare vendor quotations after RFQs have been issued. The user uploads at least two quotation files. The prototype reviews the available file text and extracts procurement fields such as vendor, product, price, delivery, warranty, support, payment terms, tax treatment, freight, installation, validity, and compliance indicators.

The comparison does not simply select the cheapest quote. It considers whether the quoted cost is clear, whether taxes and freight are included, whether delivery and warranty are stated, whether support is available, whether payment terms are acceptable, and whether the vendor has compliance or authorization signals.

The Decision Summary highlights:

- Best value
- Lowest price
- Fastest delivery
- Top clarification needed
- Main negotiation point

Each vendor card shows the practical award considerations: price, products, delivery, warranty/support, payment/validity, taxes/freight, compliance, pros, cons, risks, and negotiation levers.

Because this is a demo prototype, missing or unreadable information is not treated as a system error. It is treated as a procurement clarification item.

Final Notes

The current implementation satisfies the project requirement for a deterministic, local, demo-only quotation comparison workflow. It is suitable for enterprise prototype demonstrations where the goal is to show procurement decision support, not production-grade document ingestion.

No application functionality was changed to produce this documentation.